

Assignment 2 — Manual Calculations (Part B)

Introduction to Artificial Intelligence

Question 1 Part B — Decision Tree: Manual Information Gain Calculation

Dataset (Play Tennis — 5 examples)

#	Outlook	Humidity	Play
1	Sunny	High	No
2	Sunny	Normal	Yes
3	Overcast	High	Yes
4	Rain	High	No
5	Rain	Normal	Yes

Step 1 — Entropy of the Parent Node (entire dataset)

The parent set S has 5 examples: 3 Yes, 2 No.

Formula: $H(S) = -p(\text{Yes}) \cdot \log_2 p(\text{Yes}) - p(\text{No}) \cdot \log_2 p(\text{No})$

$p(\text{Yes}) = 3/5 = 0.6$, $p(\text{No}) = 2/5 = 0.4$

$H(S) = -0.6 \cdot \log_2(0.6) - 0.4 \cdot \log_2(0.4)$

$= -0.6 \cdot (-0.7370) - 0.4 \cdot (-1.3219)$

$= 0.4422 + 0.5288$

$H(S) = 0.9710$ bits

Step 2 — Information Gain for Outlook

Outlook has 3 values: Sunny (2 rows), Overcast (1 row), Rain (2 rows).

Sunny subset → rows 1,2 → [No, Yes] → 1 Yes, 1 No

$H(\text{Sunny}) = -0.5 \cdot \log_2(0.5) - 0.5 \cdot \log_2(0.5) = 1.0000$

Overcast subset → row 3 → [Yes] → pure → $H(\text{Overcast}) = 0.0000$

Rain subset → rows 4,5 → [No, Yes] → 1 Yes, 1 No

$H(\text{Rain}) = -0.5 \cdot \log_2(0.5) - 0.5 \cdot \log_2(0.5) = 1.0000$

Weighted Entropy = $(2/5) \cdot 1.0 + (1/5) \cdot 0.0 + (2/5) \cdot 1.0 = 0.8000$

$\text{Gain}(\text{Outlook}) = 0.9710 - 0.8000 = 0.1710$

Step 3 — Information Gain for Humidity

Humidity has 2 values: High (3 rows), Normal (2 rows).

High subset → rows 1,3,4 → [No, Yes, No] → 1 Yes, 2 No

$$H(\text{High}) = -(1/3) \cdot \log_2(1/3) - (2/3) \cdot \log_2(2/3)$$

$$= 0.5283 + 0.3900 = 0.9183$$

$$\text{Normal subset} \rightarrow \text{rows 2,5} \rightarrow [\text{Yes}, \text{Yes}] \rightarrow \text{pure} \rightarrow H(\text{Normal}) = 0.0000$$

$$\text{Weighted Entropy} = (3/5) \cdot 0.9183 + (2/5) \cdot 0.0000 = 0.5510$$

$$\text{Gain(Humidity)} = 0.9710 - 0.5510 = 0.4200$$

Step 4 — Choose Root and State Prediction Rules

$$\text{Gain(Humidity)} = 0.4200 > \text{Gain(Outlook)} = 0.1710$$

→ Humidity is chosen as the root of the decision stump.

Rules:

- Humidity = High → majority label: No (2 No, 1 Yes)
- Humidity = Normal → majority label: Yes (2 Yes, 0 No)

✓ These rules exactly match the Python program output.

Question 2 Part B — Perceptron: Written Analysis

Did the network learn AND correctly?

Yes. The perceptron learned AND correctly. By epoch 20 all four inputs were already predicted correctly, and the output for [1,1] converged to ~0.896 by epoch 100. This works because AND is linearly separable — a single straight decision boundary in the 2D input plane can cleanly divide the one positive point (1,1) from the three negative points (0,0), (0,1), (1,0).

Did the network learn XOR correctly?

No. The perceptron completely failed to learn XOR. Even after 100 epochs, all four predictions were wrong. XOR is NOT linearly separable: the two positive class points (0,1) and (1,0) sit at opposite corners of the unit square, and so do the two negative points (0,0) and (1,1). In a 2D input plane, no single straight line (the only boundary a single perceptron can draw) can separate these diagonal pairs — any line that separates one pair ends up grouping the wrong points on either side.

Geometric Illustration — Why XOR Cannot Be Separated by a Line

XOR is NOT Linearly Separable

No single straight line can separate blue from red

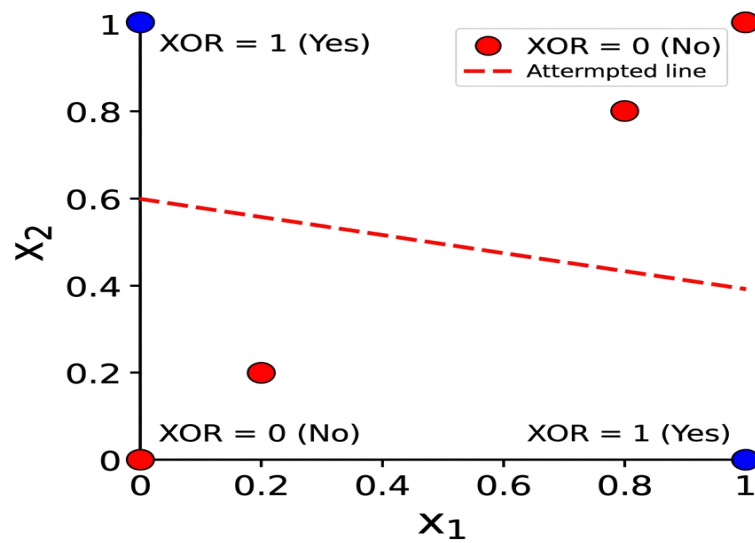


Figure: The four XOR inputs plotted in 2D. Red points (XOR=0) sit at diagonal corners (0,0) and (1,1); Blue points (XOR=1) sit at the other diagonal corners (0,1) and (1,0). The dashed line illustrates that any straight decision boundary misclassifies at least one point — no linear separator exists.

How to fix XOR (in one line):

Add a hidden layer with at least 2 neurons and a non-linear activation (sigmoid or ReLU) to form a Multi-Layer Perceptron (MLP) that can learn the non-linear XOR boundary.